

MONEYWISE

Documento de Arquitetura de Software

Equipe do projeto

Integrante	Papel
Agatha Karenne De Andrade Machado	Desenvolvedor
Ana Eduarda Raposo Medeiros	Desenvolvedor
Anette Stefany Villalba Palomino	Desenvolvedor
Brunno Calado Cavalcante	Desenvolvedor
Clarice Christine Soares Viana	Desenvolvedor
Débora Cristina Silva Ferreira	Desenvolvedor

Histórico de revisão

Data	Autor	Descrição	Versão
14/04/2026	Débora Cristina Silva Ferreira	Versão inicial	1.0

1. INTRODUÇÃO

1.1 Finalidade

Este documento fornece uma visão arquitetural abrangente do sistema MoneyWise, utilizando múltiplas visões para representar diferentes aspectos do sistema. O objetivo é capturar e comunicar as decisões arquiteturais significativas tomadas para garantir que o software atenda aos requisitos funcionais e não funcionais documentados, servindo como referência técnica para toda a equipe de desenvolvimento.

1.2 Escopo

O MoneyWise é uma aplicação mobile desenvolvida em Flutter/Dart para iOS e Android, destinada ao gerenciamento de finanças pessoais e colaborativas. O sistema permite que jovens adultos, casais e famílias registrem transações, criem grupos financeiros compartilhados, acompanhem metas coletivas por meio de um indicador visual e exportem relatórios mensais em PDF.

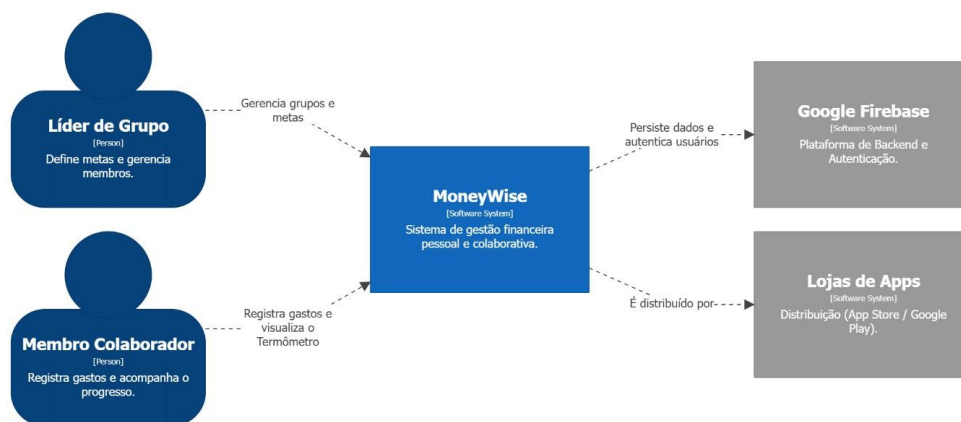
Este documento cobre:

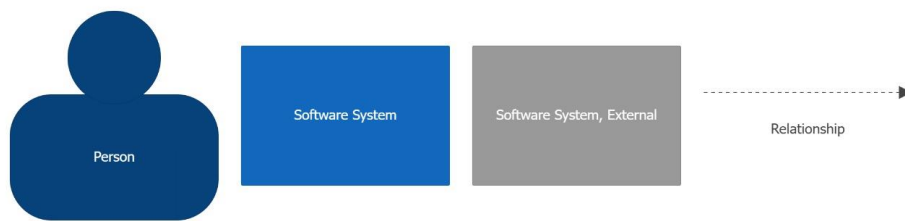
- A descrição geral da arquitetura e o estilo arquitetural adotado.
- Os principais componentes do sistema e suas responsabilidades.
- Os diagramas arquiteturais (camadas, pacotes, implantação, sequência).
- As tecnologias utilizadas e a justificativa para cada escolha.
- Os requisitos não funcionais com impacto direto na arquitetura.

1.2.1 Diagrama de Contexto (Modelo C4 – Nível 1)

Para fornecer uma visão macroscópica do ecossistema do MoneyWise, o diagrama abaixo ilustra como o sistema interage com os usuários (Líderes e Membros) e com as dependências externas fundamentais (Firebase e Lojas de Aplicativos).

- **Responsabilidade:** O MoneyWise atua como o orquestrador da saúde financeira, consumindo serviços de infraestrutura (Firebase) para garantir que os dados de transações e grupos sejam persistidos e sincronizados entre os atores.





1.3 Definições, Acrônimos e Abreviações

Termo	Definição
Flutter	Framework de desenvolvimento mobile multiplataforma da Google, utilizado para criar o cliente iOS e Android do MoneyWise.
Dart	Linguagem de programação utilizada pelo Flutter, adotada como linguagem exclusiva.
Firebase	Plataforma de backend como serviço (BaaS) do Google, utilizada como backend exclusivo.
Firestore	Banco de dados NoSQL em tempo real do Firebase, utilizado para persistência.
BLoC	Business Logic Component, padrão de gerenciamento de estado reativo utilizado na camada de lógica de negócio do cliente Flutter.
Repository Pattern	Padrão de projeto que abstrai o acesso a dados, isolando a lógica de negócio do Firebase.
FCM	Firebase Cloud Messaging, serviço de notificações push.
Caixinha	Termo do domínio que representa uma meta financeira coletiva dentro de um grupo MoneyWise.
Termômetro	Indicador visual dinâmico que representa o progresso de uma meta coletiva.
RF	Requisito Funcional descreve o que o sistema deve fazer.
RNF	Requisito Não Funcional descreve como o sistema deve se comportar.

Termo	Definição
RN	Regra de Negócio invariante de domínio que governa o comportamento do sistema.
REST	Restrição do Sistema limitação externa ou arquitetural imposta ao projeto.

2. REPRESENTAÇÃO ARQUITETURAL

A arquitetura do sistema MoneyWise segue o modelo cliente-servidor, sendo composta por um aplicativo móvel desenvolvido em Flutter e um backend baseado no Firebase. O cliente é responsável pela interface e interação com o usuário, enquanto o Firebase provê serviços como autenticação, armazenamento de dados em tempo real, notificações e controle de acesso.

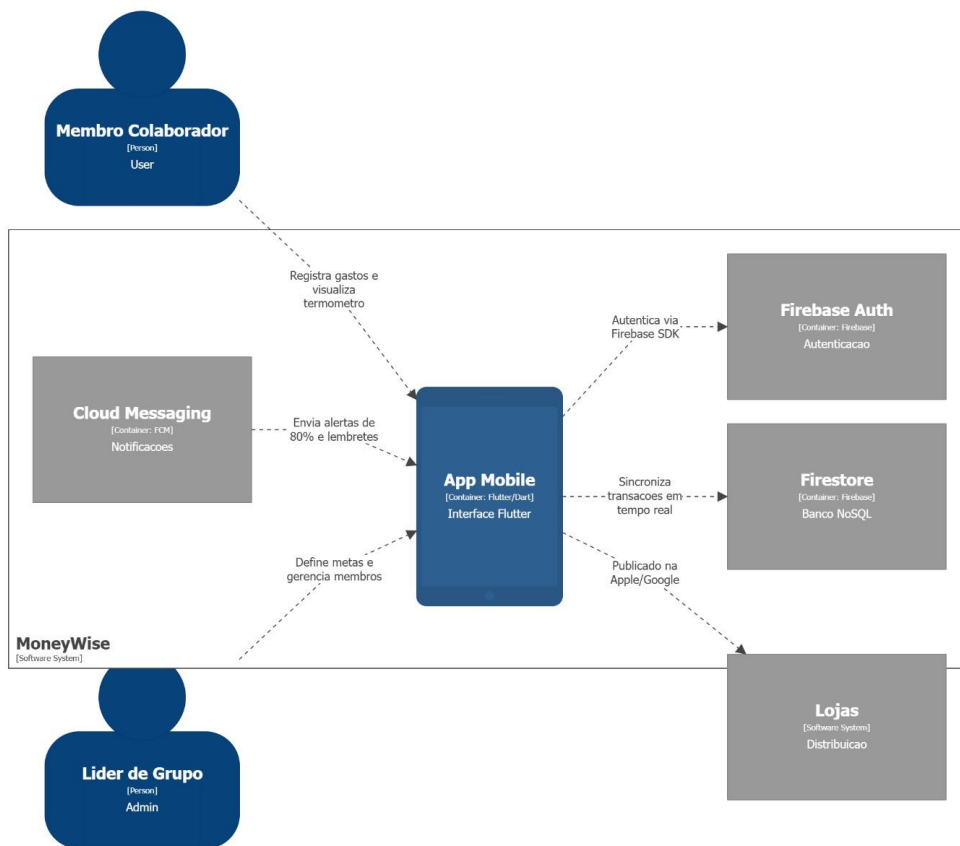
A escolha por essa arquitetura se deve à necessidade de sincronização em tempo real entre usuários, especialmente em funcionalidades que envolvem interação em grupo. O uso do Firebase simplifica essa implementação ao oferecer suporte nativo a atualizações em tempo real e reduzir a complexidade do backend.

No cliente, a aplicação é estruturada em camadas: apresentação, lógica de negócio e dados, com o objetivo de separar responsabilidades e facilitar a manutenção. Além disso, o sistema é organizado por funcionalidades, como autenticação, transações, grupos e metas, permitindo maior modularidade e escalabilidade.

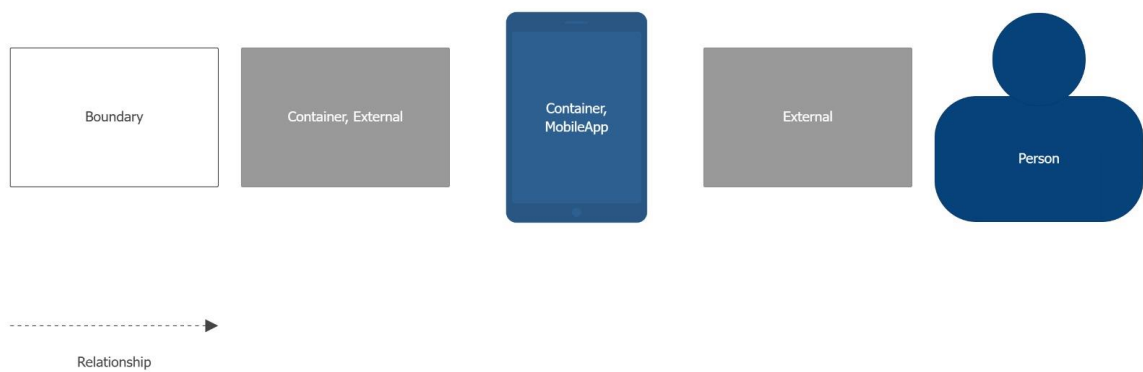
2.1. Visão de Containers (Modelo C4 - Nível 2)

Para detalhar as tecnologias previstas e suas interações, a arquitetura é decomposta em containers independentes que formam a stack tecnológica (REST-04).

- **App Mobile (Flutter):** Ponto de interação única do usuário, responsável pela lógica local e interface.
- **Firebase Cloud Messaging (FCM):** Gerencia o envio de alertas de 24h (RN-07).
- **Firebase Auth:** Centraliza a segurança de identidade.
- **Firestore NoSQL:** Garante a persistência de dados com suporte nativo a sincronização offline (RNF-02).



Container View: MoneyWise
Diagrama de Containers
sexta-feira, 10 de abril de 2026 às 15:50 Horário Padrão de Brasília



Visão	Público	Área	Descrição no MoneyWise
Lógica	Analistas e desenvolvedores	Realização dos Casos de Uso	Organização em camadas (UI, BLoC, Repository) e features (auth, transactions, groups, goals).
Processo	Integradores	Performance e Concorrência	Fluxo de dados entre cliente Flutter e Firebase; sincronização offline/online via cache nativo do Firestore.
Implementação	Programadores	Componentes de Software	Estrutura de pastas feature-first; classes BLoC, Repository e modelos Dart por módulo.
Implantação	Gerência de Configuração	Nodos físicos	Dispositivo móvel (Android 10+/iOS) executando o app Flutter; servidores Firebase gerenciados pelo Google.
Caso de Uso	Todos	Requisitos funcionais	UC01 a UC10 documentados no Documento de Requisitos v1.1.
Dados	Especialistas em dados	Persistência	Firebase Firestore com coleções users, transactions, groups e goals; Security Rules garantindo isolamento de dados (RNF-05).

3. REQUISITOS E RESTRIÇÕES ARQUITETURAIS

Esta seção descreve os requisitos de software e restrições com impacto significativo na arquitetura, mapeados diretamente dos documentos de requisitos e visão do MoneyWise.

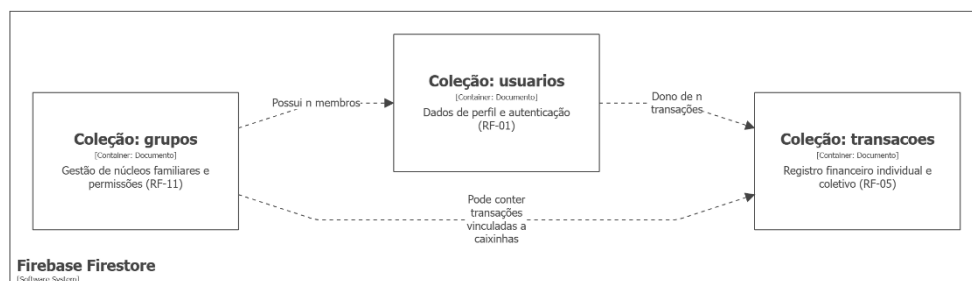
Requisito	Solução Arquitetural	Referência
Linguagem e Framework	Dart com Flutter. Não é permitido utilizar outras linguagens ou frameworks no Semestre 1.	REST-04
Plataforma mobile	Android 10+ e iOS. O Flutter compila para ambas as plataformas a partir de um único código-fonte.	RNF-06
Backend exclusivo	Google Firebase (plano gratuito). Não é permitido adicionar outro servidor ou BaaS.	REST-03
Banco de dados	Firebase Firestore com boas práticas de modelagem e Security Rules configuradas.	REST-02
Segurança	Firebase Authentication para autenticação; Security Rules no Firestore garantindo que nenhuma query retorne dados de outro usuário, mesmo com token válido.	RNF-04, RNF-05
Persistência offline	Cache de persistência nativo do Firestore, sem necessidade de banco local adicional. Sincronização em até 5 segundos após reconexão.	RNF-02, RNF-12
Sem integração bancária	Todos os dados financeiros são inseridos manualmente pelo usuário. Não há Open Banking ou sincronização com contas bancárias.	REST-01, REST-06
Internacionalização	Todos os cálculos são realizados na moeda local configurada no dispositivo do usuário (ex.: BRL para Brasil).	RF-27, RN-09

Requisito	Solução Arquitetural	Referência
Notificações push	Firestore Cloud Messaging (FCM) para envio de notificações mesmo com app em background ou fechado.	RNF-11, RF-28
Privacidade de dados	O saldo pessoal individual não é visível aos demais membros do grupo. Security Rules impedem acesso cruzado entre usuários.	RN-01, RN-04, RNF-05

3.1 Estrutura de Dados e Persistência (Firestore NoSQL)

Para viabilizar as restrições de segurança (RNF-05) e a separação entre saldo pessoal e coletivo (RN-08), a arquitetura de dados utiliza uma estrutura de coleções otimizada para o Firestore.

- Coleção usuarios: Armazena metadados de perfil e moedas preferenciais (RF-01, RN-09).
- Coleção grupos: Centraliza a gestão de núcleos familiares e permissões de Admin (RF-11, RN-06).
- Coleção transacoes: Registros financeiros individuais e coletivos, permitindo a filtragem por categoria e data (RF-05, RF-08).
- Subcoleção caixinhas: Localizada dentro de cada grupo para isolar aportes coletivos, garantindo que transações pessoais não sejam expostas indevidamente (RN-01, RN-04).



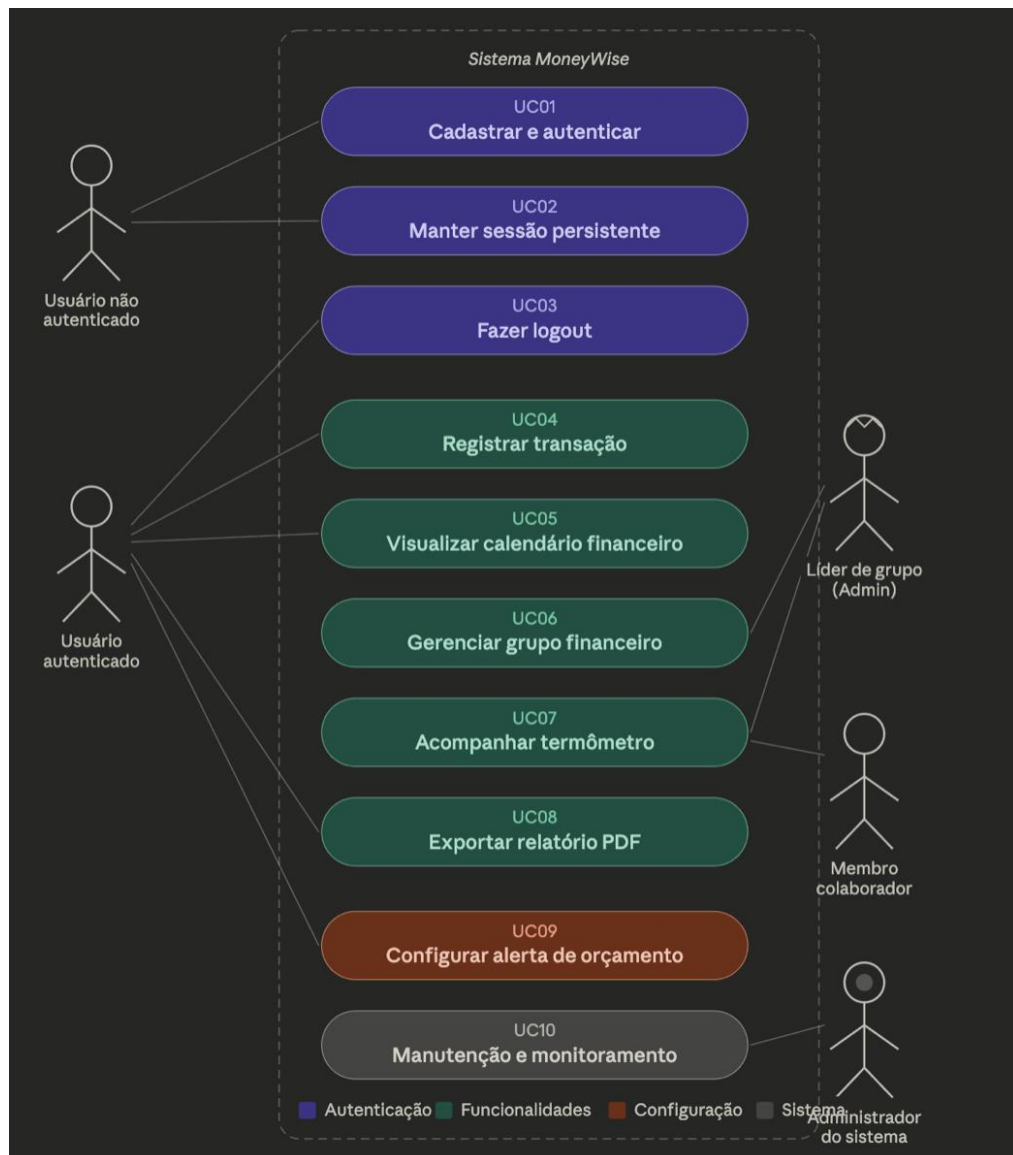


4. VISÃO DE CASOS DE USO

Esta seção lista os casos de uso significativos para a arquitetura do MoneyWise, ou seja, aqueles que influenciam decisões estruturais do sistema.

4.1 Casos de Uso Significantes para a Arquitetura

Os casos de uso a seguir foram identificados como arquiteturalmente relevantes por exercerem as principais camadas do sistema e por representarem os riscos



técnicos mais críticos do projeto:

Caso de Uso	Relevância Arquitetural
UC01 — Cadastrar e Autenticar Usuário	Exercita a integração com Firebase Auth, a persistência de sessão e a regra de unicidade de e-mail. Define o fluxo de autenticação que precede todos os demais casos de uso.
UC04 — Registrar Transação	Exercita as três camadas do cliente (UI, BLoC, Repository) e a escrita no Firestore. Valida o comportamento offline-first e o tempo de resposta máximo de 2 segundos.
UC06 — Gerenciar Grupo Financeiro	Exercita a sincronização em tempo real entre múltiplos usuários, a hierarquia de permissões Admin/Membro e o isolamento de dados pessoais.
UC07 — Acompanhar Termômetro de Meta	Exercita o motor de regras de cálculo térmico e a atualização em tempo real do indicador visual, validando a latência de sincronização do Firestore.
UC09 — Configurar Alerta de Orçamento	Exercita as regras de limite orçamentário e o sistema de notificações push via FCM, que deve funcionar com o app em background.
UC10 — Manutenção e Monitoramento	Define os requisitos de disponibilidade e as responsabilidades do administrador do sistema sobre a infraestrutura Firebase.

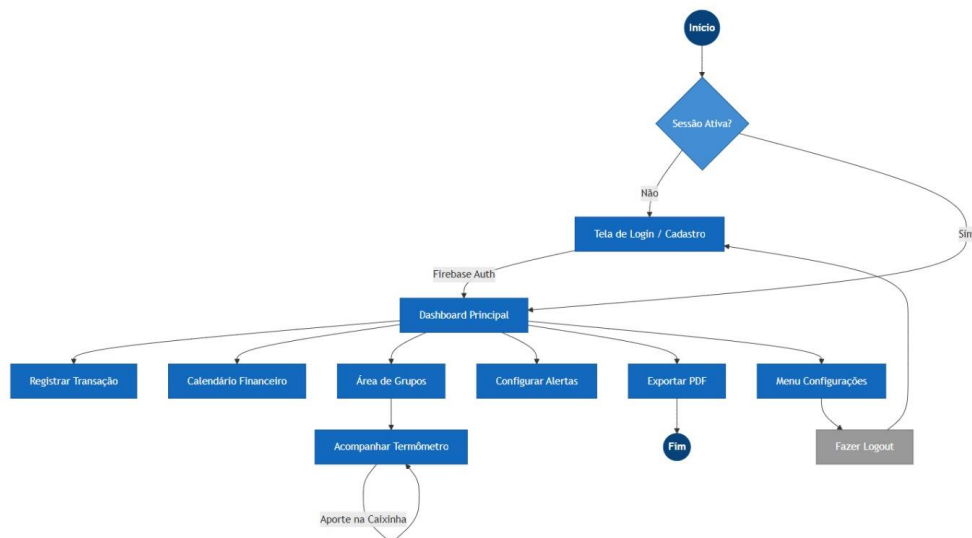
Os atores do sistema são: Usuário não autenticado (UC01 a UC03), Usuário autenticado (UC04 a UC09), Líder de grupo/Admin (UC06, UC07) e Administrador do sistema (UC10), conforme especificado no Documento de Visão v1.1.

4.2 Fluxo de Navegação e Experiência do Usuário (User Flow)

Para garantir a rastreabilidade entre os Casos de Uso (UC01 a UC10) e a interface mobile, o diagrama abaixo mapeia os estados do aplicativo e os pontos de decisão arquitetural, como a validação de sessão ativa.

- **Fluxo de Entrada:** Gerencia a transição entre o **Usuário Não Autenticado** (UC01) e o estado de **Sessão Persistente** (UC02).
- **Fluxo Operacional:** A partir do **Dashboard Principal**, o usuário acessa de forma modular o registro de transações (UC04) ou a gestão de grupos (UC06).
- **Fluxo de Engajamento:** Detalha o ciclo de feedback do **Acompanhar Termômetro** (UC07) e a configuração de alertas (UC09).

- **Fluxo de Encerramento:** Cobre a exportação de **Relatórios PDF** (UC08) e o encerramento seguro da sessão (UC03).



5. VISÃO LÓGICA

A visão lógica descreve como o sistema MoneyWise está organizado internamente, detalhando as camadas, pacotes e a responsabilidade de cada componente.

5.1 Arquitetura em Camadas — Visão Geral

O cliente Flutter é organizado em três camadas horizontais, aplicadas verticalmente por feature:

Camada	Responsabilidade	Exemplos no MoneyWise
Apresentação (UI)	Renderizar a interface e capturar interações do usuário. Não contém lógica de negócio.	LoginPage, TransactionPage, GroupPage, GoalsPage, ThermometerWidget.
Lógica de Negócio (BLoC)	Receber eventos da UI, aplicar regras de negócio e emitir estados. Não acessa o Firebase diretamente.	AuthBloc, TransactionBloc, GroupBloc, GoalBloc
Dados (Repository)	Abstrair o acesso ao Firebase Firestore e Firebase Auth.	AuthRepository, TransactionRepository, GroupRepository, GoalRepository

Camada	Responsabilidade	Exemplos no MoneyWise
	Converter modelos Dart para/do formato Firestore.	

5.2 Organização por Feature — Estrutura de Pacotes

O código é organizado por feature (feature-first), garantindo que cada módulo seja coeso e independente:

Pacote / Feature	Conteúdo	Requisitos Atendidos
lib/core/	Entidades compartilhadas, utilitários, tratamento de erros, router de navegação e injeção de dependência.	Todos os módulos
lib/features/auth/	Telas de login/cadastro, AuthBloc, AuthRepository, modelo User. Integra com Firebase Authentication.	RF-01 a RF-04, RN-02, RN-06
lib/features/transactions/	Telas de transações, TransactionBloc, TransactionRepository, modelo Transaction. Persistência no Firestore com suporte offline.	RF-05 a RF-10, RN-05
lib/features/groups/	Telas de grupos, GroupBloc, GroupRepository, modelo Group. Sincronização em tempo real; controle de acesso Admin/Membro.	RF-11 a RF-16, RN-01, RN-04, RN-06
lib/features/goals/	Tela do Termômetro, GoalBloc, GoalRepository, modelo Goal. Motor de cálculo térmico e alertas visuais.	RF-17 a RF-20, RN-03, RN-07
lib/features/reports/	Calendário financeiro, dashboards, geração de PDF mensal.	RF-21 a RF-24, RF-26, RN-05
lib/features/settings/	Tema claro/escuro, configuração de moeda, notificações push.	RF-25, RF-27, RF-28, RN-07, RN-09

5.3 Regra de Dependência

A regra fundamental da arquitetura é: a camada de Apresentação depende do BLoC, o BLoC depende do Repository, e o Repository depende do Firebase. Nenhuma camada superior é conhecida pelas camadas inferiores. A UI nunca acessa o Firestore diretamente.

Fluxo de dados: UI → BLoC → Repository → Firebase Firestore/Auth

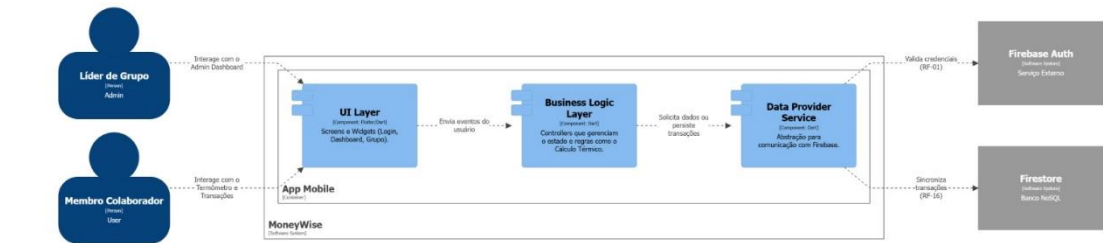
Fluxo de resposta: Firebase → Repository → BLoC (emite estado) → UI
(BlocBuilder reconstrói a tela)



5.3.1 Detalhamento da Arquitetura Interna (Modelo C4 - Nível 3)

Para complementar a regra de dependência citada no item anterior, o diagrama abaixo detalha a visão interna do App Mobile, evidenciando como os componentes de UI, Lógica e Dados se integram e realizam as comunicações externas.

- UI Layer: Composta por Screens e Widgets que capturam eventos do usuário e exibem o estado final do sistema.
- Business Logic Layer: Centraliza o gerenciamento de estado e executa as regras críticas de domínio, como o Cálculo Térmico (RN-03).
- Data Provider Service (Repository): Atua como a abstração para comunicação com o Firebase Auth (RF-01) e Firestore (RF-16), garantindo que a lógica de negócio permaneça agnóstica à fonte de dados.



Component View: MoneyWise - App Mobile
 Arquitetura Interna Flutter
 sexta-feira, 10 de abril de 2020 às 15:53 Horário Padrão de Brasília



6. VISÃO DE IMPLEMENTAÇÃO

Esta seção descreve como os casos de uso são realizados pelos componentes de software, ilustrando a interação entre as camadas.

6.1 UC04 — Registrar Transação

6.1.1 Diagrama de Classes (descrição textual)

As principais classes envolvidas no registro de uma transação são:

Classe	Camada	Responsabilidade
TransactionPage	Apresentação	Widget Flutter que exibe o formulário de cadastro e dispara o evento AddTransactionEvent ao confirmar.
TransactionBloc	BLoC	Recebe AddTransactionEvent, chama TransactionRepository.save(), emite TransactionLoadingState e depois TransactionSuccessState ou TransactionErrorState.
TransactionRepository	Dados	Método save(Transaction t): converte o modelo para Map via toJson() e persiste no Firestore na coleção /transactions/{userId}.
Transaction	Modelo	Classe Dart pura com campos: id, userId, valor, data, categoria, tipo (despesa/receita). Métodos: fromJson(), toJson().
FirebaseFirestore	Infraestrutura	SDK do Firestore. Persiste o documento e, se offline, enfileira para sincronização automática.

6.1.2 Diagrama de Sequência (descrição textual)

O fluxo de registro de uma transação segue a seguinte sequência:

- 1. O usuário preenche os campos (valor, data, categoria) na TransactionPage e toca em 'Salvar'.
- 2. TransactionPage dispara AddTransactionEvent com os dados preenchidos para o TransactionBloc.
- 3. TransactionBloc emite TransactionLoadingState (UI exibe indicador de carregamento).

- 4. TransactionBloc chama TransactionRepository.save(transaction).
- 5. TransactionRepository serializa o modelo (toJson()) e chama FirebaseFirestore.collection('transactions').add(data).
- 6. Se online: Firestore confirma a escrita. Se offline: Firestore enfileira a operação e confirma localmente (cache persistente).
- 7. TransactionRepository retorna sucesso para o TransactionBloc.
- 8. TransactionBloc emite TransactionSuccessState com o saldo atualizado.
- 9. BlocBuilder na TransactionPage reconstrói a UI exibindo confirmação e saldo atualizado.

Tempo máximo de resposta: 2 segundos (RNF-01). Em caso de ausência de internet, a resposta local é imediata e a sincronização ocorre em até 5 segundos após reconexão (RNF-12).

6.2 Realização do Caso de Uso Crítico: Sincronização do Termômetro (UC06/UC07)

A funcionalidade de acompanhamento de metas em grupo é o diferencial técnico do sistema. Para garantir a RN-03 e a sincronização em tempo real (RF-16), a arquitetura utiliza um fluxo reativo detalhado no diagrama abaixo.

6.2.1 Diagrama de Sequência: Atualização Reativa do Termômetro

Este diagrama ilustra como um aporte realizado por um membro reflete instantaneamente para todo o grupo, executando o cálculo térmico antes da renderização.

- **Processamento Local:** O cálculo da razão entre valor acumulado e prazo restante (RN-03) ocorre na camada de lógica (BLoC) assim que o *snapshot* do Firestore é recebido.
- **Feedback Visual:** A interface reage ao novo estado emitido, atualizando o indicador visual (RF-17) em menos de 2 segundos (RNF-01).

servidor garantem que dados de grupos e caixinhas sejam inacessíveis a não membros.

```
// Exemplo de regra lógica aplicada à arquitetura
service cloud.firestore {
  match /databases/{database}/documents {
    match /grupos/{grupoId} {
      // Permite leitura se o usuário for membro do grupo
      allow read: if request.auth.uid in resource.data.membros;
      // Permite escrita na meta apenas se for o Admin
      allow update: if request.auth.uid ==
resource.data.id_admin;

      match /caixinhas/{caixinhaId} {
        allow read: if request.auth.uid in
get(/databases/{database}/documents/grupos/{grupoId}).data.membros;
      }
    }
  }
}
```

7. VISÃO DE IMPLANTAÇÃO

Esta seção descreve os nodos físicos e a distribuição dos artefatos de software no ambiente de produção.

Nodo	Tipo	Artefatos implantados	Responsável
Dispositivo do usuário (Android 10+)	Smartphone Android	Aplicativo MoneyWise compilado (.apk/.aab), cache local do Firestore (offline)	Google Play Store (REST-07)
Dispositivo do usuário (iOS)	iPhone/iPad	Aplicativo MoneyWise compilado (.ipa), cache local do Firestore (offline)	App Store (REST-07)
Firebase Authentication	Servidor Google (nuvem)	Serviço de autenticação; tokens JWT dos usuários	Google Firebase
Firebase Firestore	Servidor Google (nuvem)	Coleções: users, transactions, groups, goals; Security Rules	Google Firebase
Firebase Cloud Messaging	Servidor Google (nuvem)	Serviço de envio de notificações push	Google Firebase

O modelo de implantação segue a arquitetura Cliente-Servidor: o cliente (app Flutter) é distribuído pelas lojas oficiais e executa no dispositivo do usuário, enquanto todo o backend é gerenciado pela infraestrutura do Google Firebase, eliminando a necessidade de um servidor próprio e respeitando a restrição.

8. DIMENSIONAMENTO E PERFORMANCE

8.1 Volume

Estimativas de volume para o contexto acadêmico do projeto (fase de validação):

Indicador	Estimativa	Justificativa
Número estimado de usuários	50 a 200 usuários	Projeto acadêmico em fase de validação com usuários de teste e avaliadores.
Número estimado de acessos diários	100 a 500 sessões/dia	Estimativa baseada em 2 a 3 acessos diários por usuário ativo.
Número estimado de transações por usuário/mês	30 a 100 transações	Média de 1 a 3 registros de gasto por dia por usuário.
Tempo médio de sessão	5 a 10 minutos	Uso típico para registro de transações e consulta de saldo/metadados.
Volume de dados por usuário/mês	Menos de 1 MB	Transações e grupos são dados textuais simples (valor, data, categoria).

8.2 Performance

Requisito	Meta	Solução Técnica	Referência
Tempo de salvamento de registro	Menos de 2 segundos	Escrita direta no Firestore via SDK Flutter com confirmação local imediata (offline cache).	RNF-01
Sincronização após reconexão	Menos de 5 segundos	Firestore SDK enfileira operações offline e sincroniza automaticamente ao detectar conectividade.	RNF-12

Requisito	Meta	Solução Técnica	Referência
Atualização do Termômetro	Tempo real	Listener de snapshot em tempo real no Firestore (onSnapshot), sem necessidade de polling.	RF-19
Disponibilidade do sistema	99,9% de uptime	Infraestrutura gerenciada pelo Google Firebase com SLA de disponibilidade global.	RNF-07
Notificações push	Entrega mesmo com app fechado	Firebase Cloud Messaging com suporte a notificações em background em iOS e Android.	RNF-11

9. QUALIDADE

Esta seção enumera os atributos de qualidade de software significativos para o MoneyWise e como a arquitetura os endereça.

Atributo	Descrição	Solução Arquitetural	Referência
Segurança	Proteção dos dados financeiros e pessoais dos usuários.	Firebase Authentication com tokens JWT; Firebase Security Rules que impedem acesso cruzado entre usuários mesmo com token válido; dados trafegados exclusivamente via HTTPS.	RNF-04, RNF-05
Disponibilidade / Confiabilidade	O sistema deve estar disponível 99,9% do tempo.	Infraestrutura Firebase gerenciada pelo Google com redundância global. Cache offline garante uso sem internet (RNF-02).	RNF-07, RNF-02
Portabilidade	Suporte a Android 10+ e iOS a partir de um único código-fonte.	Flutter compila para ambas as plataformas nativamente, seguindo Material Design e padrões iOS (RNF-03).	RNF-06, REST-04
Manutenibilidade	Código legível, comentado e padronizado para facilitar manutenção pela equipe.	Organização feature-first com responsabilidades bem definidas por camada (BLoC, Repository); código comentado em português seguindo padrões Dart (RNF-08).	RNF-08
Escalabilidade / Suportabilidade	Adição de novos módulos sem reescrita das telas existentes.	Arquitetura feature-first permite adicionar novas features (ex.: conversão de moedas) sem alterar módulos existentes. BLoC isola estado por feature.	RNF-09
Testabilidade	Fluxos principais testáveis sem emulador ou dispositivo físico.	BLoC e Repository são classes Dart puras que podem ser testadas com mocks do Firebase, sem necessidade de ambiente real.	RNF-10

Atributo	Descrição	Solução Arquitetural	Referência
Privacidade	Dados pessoais não são expostos indevidamente entre membros do grupo.	Security Rules no Firestore garantem que cada usuário só lê/escreve seus próprios dados. Saldo pessoal nunca é compartilhado (RN-01, RN-04).	RN-01, RN-04, RNF-05